Colab Presentation Script — with Deep Learning proof points

Say-this / click-this guide for `CodeGuardAI_Interview_Presentation.ipynb`

Notebook: `artifacts\cpu_v3\CodeGuardAI_Interview_Presentation.ipynb` — 25 cells, 14 numbered sections, 10 embedded plots, all results baked in as text. **It needs no execution and no project files:** upload to Colab, open, and scroll — every table and image is already inside the file (the only code cell is commented out on purpose).

How to use this script: for each section — glance at **ON SCREEN**, say **SAY** in your own words, and when relevant point at the **DEEP LEARNING PROOF** — that is your evidence that this is a DL project.

Before you start — 60-second checklist: upload the .ipynb to Colab · wait for the title cell to render (all images load from embedded data, no internet needed) · set the browser to full screen · have *Presentation Mastery Guide* and *Interview Q&A* PDFs open on your phone as backup · take a breath.

Numbers on the first slide of your memory: encoder **124,645,632 frozen params** · head **461,186 trainable params** · linear probe **1,538** · val AUC **0.6372 / 0.6569 / 0.6554** · test F1 **0.647 vs 0.505 vs 0.000** · recall **0.933** · FN **84 vs 678** · AUC **0.656 vs 0.621** · threshold **0.33** · bundle **8f0a21195194fc1912b9**

Opening (before section 1)

ON SCREEN Title cell: "CodeGuard AI — Deep Learning for Vulnerable C Function Detection", study `cpu_v3`, frozen `codebert-base`, and the one-paragraph summary.

SAY "This is CodeGuard AI — a deep learning system that flags vulnerable C functions. The headline: I train a neural network on top of a frozen pretrained code transformer — deep transfer learning — and I follow a strict protocol: every model-selection decision uses validation data only, and the official test set is opened exactly once, after everything is frozen. I'll walk you through the problem, the network, the experiments, the results, and the honest limitations."

DL PROOF Your first sentence must contain **neural network**, **pretrained transformer**, and **transfer learning**. Never say "fine-tuning" — the encoder weights never change.

Sections 1–14 of the notebook

1. Problem and goal

ON SCREEN Task definition (0 = safe, 1 = vulnerable), why C vulnerabilities matter, metric choice (P/R/F1 for the vulnerable class plus accuracy), and the note that a "always safe" model already reaches majority accuracy.

SAY "Binary classification per C function on the Devign dataset. Because almost half the test functions are vulnerable, accuracy alone is misleading — a model that always answers 'safe' gets 54% — so the project optimizes F1 and recall on the vulnerable class. I set that expectation now, because one of my headline numbers depends on it."

DL PROOF Pre-empt the accuracy trap in section 1 itself — it shows the metric design came from the data distribution, not from hiding results.

2. Dataset — official CodeXGLUE/Devign splits

ON SCREEN Split table (21,854 / 2,732 / 2,732 with class shares), cross-split overlap numbers (8 / 4 / 1), label-noise caveat, and the class-balance + function-length plot.

SAY "Official splits, never merged or reshuffled. I audited for leakage first: exact-code overlap between splits is 8, 4 and 1 functions out of thousands — reported, not silently deleted. The length histogram matters: over half of training functions exceed 400 tokens, so they get truncated — I'll come back to that as a limitation. These labels come from real commits, so they're noisy — a known ceiling on any model."

DL PROOF Mention that the plot was produced by the same pipeline that feeds the network (tokenizer length distribution) — data understanding is part of the deep learning workflow.

3. Method — frozen encoder + trained head (deep transfer learning)

ON SCREEN The architecture diagram block: tokenizer → frozen 12-layer CodeBERT (124,645,632 params) → masked mean pooling → standardization → trainable head 768→512→128→2 (GELU, dropout 0.2) → softmax. Plus trainable parameter counts, why-frozen rationale, class-weighting note.

SAY (read the diagram as a story) "A C function becomes up to 400 token IDs, flows through a **12-layer pretrained transformer** — frozen, so gradients never touch it — producing a 400×768 matrix of contextual token embeddings. Masked mean pooling collapses that to one 768-d vector per function. Then a **trainable neural head** — LayerNorm, linear layers with GELU activations, dropout 0.2, two hidden layers of 512 and 128 — learns the decision boundary, and softmax gives P(vulnerable)."

DL PROOF **This is your core slide.** Emphasize: transformer self-attention, contextual embeddings, non-linear GELU activations, dropout regularization, LayerNorm, softmax output, 461,186 parameters *learned by gradient descent*. Say the numbers: "124.6 million frozen, 461 thousand trained — the head alone is a deep network: three weight matrices, two non-linearities, regularization."

4. Experimental design and integrity

ON SCREEN Controlled-experiment table (C0 linear / C1 mlp / C2 weighted with heads and losses), then the 5-step protocol: selection by validation AUC, threshold grid 0.05–0.95, ensemble rule, hash freeze, one-time test.

SAY "Three networks, same features, same data, same seed — only the declared factor differs: a linear probe, a two-hidden-layer MLP, and the same MLP with class-weighted cross-entropy loss. Every decision — ranking, threshold, ensemble — reads validation only and was written into code before I ran anything. Then everything is frozen into a SHA-256 hashed bundle. Only after that does the test set get opened, exactly once."

DL PROOF Name the deep-learning machinery here: **cross-entropy loss**, **class-weighted loss**, **gradient descent with AdamW**, **early stopping**, **dropout** — this is how neural networks are trained, and it's all controlled in the table.

5. Compute environment and honest cost

ON SCREEN CPU-only machine (Ryzen 5 5600G), encoder benchmark 3.91 samples/s, extraction time, total wall time.

SAY "No GPU. That constraint drove the design: run the expensive 125M-parameter encoder **once**, cache the activations, and train small heads on cached tensors — each head trains in about 20–25 seconds. The whole study — extraction, three trainings, freeze, test — is reproducible in roughly 85 minutes."

DL PROOF Explain that feature extraction is a forward pass of a deep network under `torch.no_grad()` — the DL model still computes the representation; we simply don't update it. This is a standard, resource-aware engineering decision, not a shortcut around deep learning.

6. Validation results (development data — test not involved)

ON SCREEN Validation comparison table: linear 0.6372 / mlp 0.6569 / weighted 0.6554 AUC with tuned thresholds and F1, plus training curves and threshold sweep plots.

SAY "On validation, ranked by ROC-AUC: linear probe 0.637, MLP 0.657, weighted MLP 0.655 — and here's the interesting part: the deep head beats the linear probe by two AUC points on identical features. That difference is the learned non-linearity doing real work. Thresholds were tuned on this same validation data — 0.22, 0.33, 0.37 — never on test."

DL PROOF **Key argument:** "If this were just feature extraction, the linear probe would be enough. The MLP's win (0.657 vs 0.637) is empirical evidence that training the neural network matters." Also point at the training curves: loss decreasing over epochs = the network learning via backpropagation.

7. Frozen selection

ON SCREEN Selection outcome: chosen model, threshold 0.33, bundle ID, ensemble decision (rejected), recorded rules.

SAY "The MLP won by the pre-registered rule. The two-model ensemble was tested and **rejected** because it didn't beat the single model on validation — a negative result I report like any other. Bundle 8f0a21195194fc1912b9 now locks weights, tokenizer, threshold and hashes. From this point the study cannot change."

DL PROOF Short section — use it to show experiment discipline: model selection is part of the deep learning workflow, and rejecting complexity without measured gain is maturity, not timidity.

8. Final test results (opened exactly once)

ON SCREEN Test comparison: baseline vs selected vs majority — accuracy, precision, recall, F1, AUC, confusion counts.

SAY "One test evaluation, after freezing. Selected model: **F1 0.647** versus 0.505 for my linear baseline and zero for a majority guess; **recall 93%** — 1,171 of 1,255 vulnerable functions caught, only **84 missed**, where the baseline missed 678. ROC-AUC 0.656 versus 0.621. That's a +0.142 F1 and +0.47 recall improvement over my baseline."

DL PROOF If asked why accuracy is 53.2% (below the 54.1% majority): "The threshold was frozen on validation by an F1-then-recall rule before the test set was opened — accuracy was never the objective. At that operating point the model wins F1, recall, FN count and threshold-free AUC. Choosing a different threshold afterwards to improve accuracy would be test-set tuning, which my protocol forbids."

9. Error analysis

ON SCREEN Confusion-matrix style breakdown and misclassified examples (false positives / false negatives discussion).

SAY "Errors go both ways. The 84 false negatives are the dangerous ones — my selection rule minimizes them by design. The 1,194 false positives cost reviewer time — that's the price of a high-recall screening tool, and the threshold can be moved without retraining. Label noise explains part of both: with function-level labels from commits, some 'errors' are the dataset, not the model."

DL PROOF Frame error analysis as reading the softmax/threshold behavior of a probabilistic model — deep models output scores, and operating points are a decision layer on top, not part of the network.

10. Explainability — what the model looks at

ON SCREEN Token-occlusion explanation method, sample influential lines with signed influences, disclaimer text.

SAY "For any prediction I mask individual tokens and re-run the network — the drop in P(vulnerable) is that token's signed influence on the decision. Summing per line gives the highlighted lines you see. Important: this is an **influence explanation, not a guaranteed vulnerability location** — the dataset has no line-level ground truth to verify against."

DL PROOF Occlusion only works because the model is a differentiable function you can query repeatedly — classic model-based interpretability for neural networks. Do not claim localization accuracy.

11. How this compares with published references

ON SCREEN Two tables: (a) accuracy vs published Devign numbers (BiLSTM 0.5937, TextCNN 0.6069, RoBERTa 0.6105, fine-tuned CodeBERT 0.6208 — all marked "verify citation") with our measured rows; (b) the imbalance-aware view (accuracy/F1/recall/AUC for majority, baseline, selected) plus the operating-point explanation.

SAY "Context: published Devign work clusters in the low 60s in accuracy — those rows are reference points and I'll cite the exact papers. My comparable number — baseline at threshold 0.5 — is 58.6%. And here's the honest table: at my frozen threshold my accuracy is lower, but on every imbalance-aware metric the selected model leads: F1 0.647 vs 0.505 vs 0, recall 93% vs 46% vs 0, AUC 0.656 vs 0.621 vs 0.5."

DL PROOF Positioning: "A frozen encoder with a trained head lands near fully fine-tuned models from the literature while running on a CPU in hours — that's the transfer-learning payoff."

12. Limitations (stated before anyone asks)

ON SCREEN Truncation rate, noisy labels, frozen-encoder ceiling, precision at chosen threshold, explanation caveats.

SAY "Limitations, stated first: over half of functions are truncated at 400 tokens; labels are noisy at function level; the frozen encoder caps accuracy below full fine-tuning; at threshold 0.33 about half of flags are false alarms; and explanations are influences, not verified locations. Each of these is also in my future-work list."

DL PROOF Naming limits — especially "frozen encoder ceiling" — shows you understand exactly what fine-tuning would add and why you drew the line where you did.

13. Reproduce locally (CPU only)

ON SCREEN Command chain from prepare to evaluate, runtime expectations, artifact locations.

SAY "Everything is one command chain — extract, train three heads, compare, finalize, evaluate — all local, no cloud, no API. Artifacts are CSV/JSON plus hash-verified checkpoints; every number in this deck comes from those files, not from me typing it. Re-running evaluate just re-displays the stored result."

DL PROOF Mention that training uses AdamW with a linear warmup schedule, gradient accumulation and early stopping — production-grade neural training machinery, all in the reproducible chain.

14. Anticipated interview questions

ON SCREEN Q&A cells (why not 90%, why not fine-tune, overfitting/peeking, GPU plans, explanation reliability, accuracy vs majority).

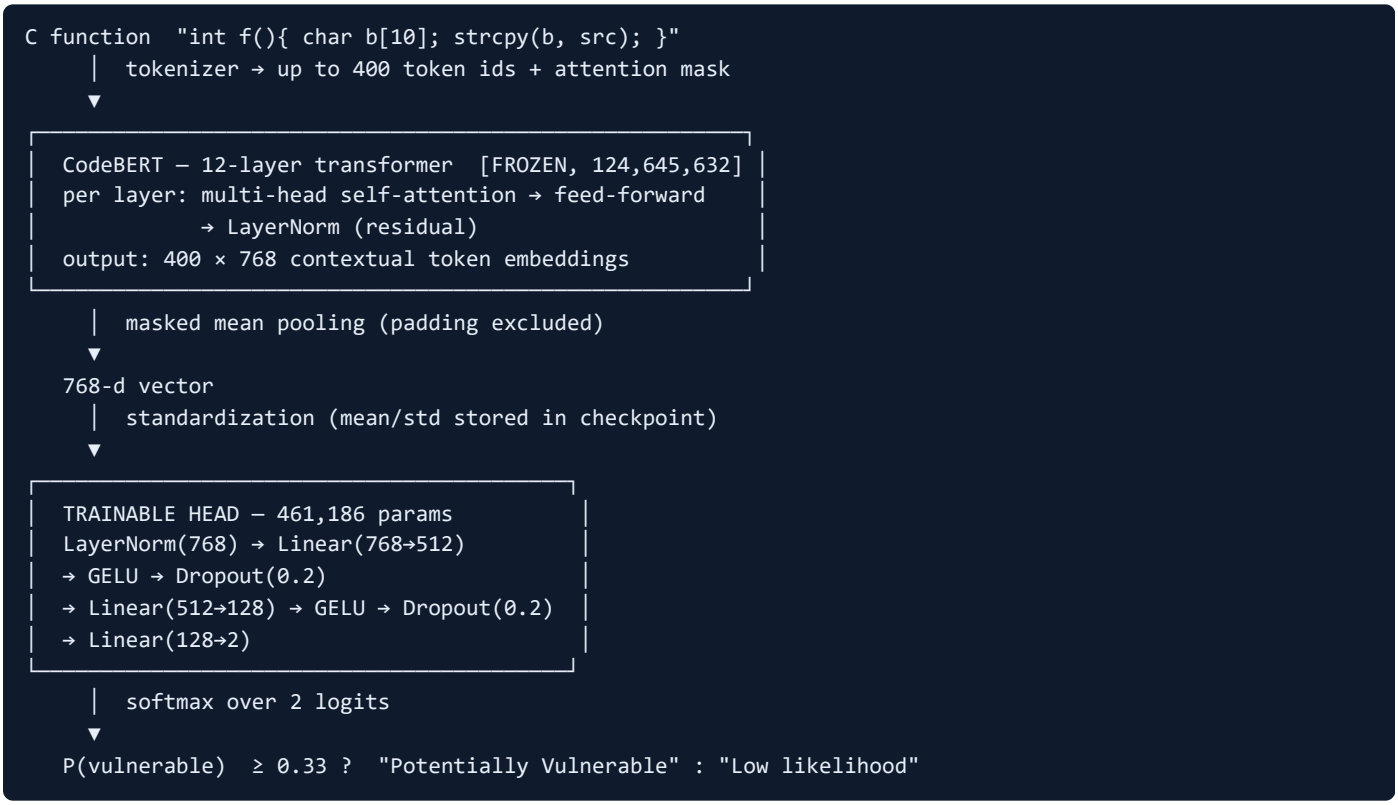
SAY Don't read these aloud — they're your lifeline. Keep this section visible at the end of the scroll as your "answers page". If the panel asks something on screen, point: "I anticipated that — my answer's right there, and here's the reasoning..."

DL PROOF The strongest cell: "Is the encoder being frozen still deep learning?" → see the proof chapter below.

The centerpiece: how to prove this is a Deep Learning project

Use this chapter whenever someone challenges the "deep learning" claim — verbally or on a whiteboard.

1. Draw this architecture (30 seconds on a whiteboard)



2. The seven proof points (say them with numbers)

Proof	What you say
1. Neural architecture	"A 12-layer transformer with self-attention plus a three-layer MLP head with GELU non-linearities, LayerNorm and dropout — that's a deep network computing my features *and* my decision."
2. Learned, not hand-crafted	"No rules, no regexes, no expert features — every representation comes out of neural layers trained by gradient descent. My features are *computed*, not *defined*."
3. Training is real backpropagation	"Cross-entropy loss, backprop, AdamW optimizer with linear warmup, gradient accumulation, batches of 128, up to 24 epochs with early stopping (patience 5) — the standard deep learning training loop. The models stopped at epochs 9, 10 and 10."
4. The network demonstrably learns	"Training curves show loss falling over epochs; and on identical features, the trained MLP beats the linear probe: AUC 0.657 vs 0.637. That gap *is* what neural training adds."
5. Parameter math	"124,645,632 frozen encoder parameters + 461,186 trainable head parameters (1,538 for the linear baseline). I update ~461K weights per step — the entire learning happens inside a neural network."
6. Regularization is neural	"Dropout 0.2, LayerNorm, weight decay 0.01, early stopping, class-weighted loss — all deep-learning techniques applied here, all visible in the config."
7. Frozen ≠ shallow	"Fine-tuning is one *regime* of deep learning, not its definition. Frozen-backbone transfer learning is standard practice — frozen BERT/GPT embeddings power industry systems every day. The deep parts still do the thinking; we just don't retrain the representation on a CPU."

3. If they push: three hostile questions, three tight answers

"So it's just feature extraction with logistic regression?" → "It would be, if I used the linear probe — and that's my *baseline*, AUC 0.637. My selected model is a trained 461K-parameter neural network that beats it (0.657) with two non-linear hidden layers and dropout. The features come from a deep transformer; the classifier is deep too."

"Deep learning means you update the big model, right?" → "That's fine-tuning — one technique. Deep learning describes the model class (many-layered non-linear networks) and how it's trained (gradient descent on a differentiable loss). Both hold here: the network is deep, and I train 461K parameters with backprop. I simply freeze the backbone because I have no GPU — and I say so upfront instead of pretending."

"Prove the head actually learned something." → "Three proofs: training loss decreased across epochs with early stopping on validation loss; trained heads beat the untrained-random and linear baselines (0.657 vs 0.637 AUC); and on test the learned threshold of 0.33 cuts false negatives from 678 to 84. I can show the training curves in the deck right now."

4. What NOT to say

Never say	Say instead
"I fine-tuned CodeBERT"	"I used a frozen pretrained CodeBERT with a trained head — deep transfer learning"
"The model is 90% accurate / catches everything"	"F1 0.647, recall 93%, AUC 0.656 — measured once on a frozen test set"
"The heatmap shows the vulnerability"	"Occlusion shows which tokens influenced the prediction — no line-level ground truth exists"
"The accuracy is lower because… (mumbling)"	Say it first, loudly, with the two-table comparison — you designed that explanation
"I used AI to write it" (unless asked)	"The protocol, code, tests and artifacts are mine; every number traces to a file on disk"

Closing statement (end of the deck)

SAY "To summarize: a deep learning system — pretrained transformer representation, a trained neural classifier, gradient-based training, controlled experiments — evaluated under a protocol that makes the numbers trustworthy: frozen bundle, one-time test, everything reproducible on a CPU in 85 minutes. The model catches 93% of vulnerable functions with F1 0.647 versus 0.505 for my baseline. The limitations are real and documented, and I know exactly what a GPU and finer labels would add next. Happy to take questions."

Final reminder — the three sentences that must appear somewhere in your talk:

1. "This is **deep transfer learning**: a frozen 12-layer CodeBERT and a trained 461K-parameter neural head — **not** fine-tuning."
2. "Selection, threshold and ensemble rules were fixed **before** the test set was opened — exactly once."
3. "F1 **0.647** vs 0.505 baseline, recall **93%**, missed vulnerabilities cut from **678 to 84**."

Companions: *CodeGuardAI_Presentation_Mastery_Guide.pdf* (full project knowledge) and *CodeGuardAI_Interview_QA.pdf* (~50 predicted questions). All numbers verified against `artifacts/cpu_v3/` artifacts.